JointShocks R Package

Johnson Oliyide and Thomas Wiesen

Contents

Purpose 1

Installation Procedure 2

Loading your own data . . . 2

Sample data used in this vignette . . . 2

Empirical Analysis 3

Impulse responses and variance decompositions . . . 3

Spillover metrics . . . 12

Saving results 24

References 24

Purpose

Economic interdependence and financial globalization have intensified the need for accurate measures of shock transmission across macroeconomic and financial systems. To this end, the **JointShocks** R package provides researchers and practitioners with robust tools to estimate the joint spillover metrics proposed in Lastrapes and Wiesen (2021) and Lastrapes and Wiesen (2025).

Additionally, it offers functionalities to compute the joint impulse response function and the joint forecast error variance decomposition introduced in Wiesen and Beaumont (2024).

For comparative purposes, the package also includes:

- The **generalized spillover index** and metrics of Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014), and
- The **generalized impulse response function** and **generalized forecast error variance decomposition** of Koop et al. (1996) and Pesaran and Shin (1998).

The package computes these measures for both the full sample and rolling windows, providing reproducible tools for empirical work in macroeconomics, finance, and banking.

The tutorial covers installation, data preparation, impulse responses, variance decompositions, bootstrapping, and spillover measures. The examples use the bundled bank volatility data.

Installation Procedure

To begin using the **JointShocks** package, install it from GitHub using the `devtools` package (only needed once):

```
install.packages("devtools")
devtools::install_github("JohnsonOliyide/JointShocks")
```

Then load it as follows:

```
library(JointShocks)
```

Because the package relies on vector autoregressions, be sure to load the `vars` package:

```
library("vars")
```

Loading your own data

Before estimating the VAR, order observations chronologically and ensure that the model variables are numeric and contain no missing values. Keep the date vector aligned with the observations. Consider whether transformations such as returns or differences are appropriate to obtain stationary model inputs.

The package currently supports data files in `.csv` or `.xlsx` format. The first column must represent dates. To read `.xlsx` files, load the `readxl` package:

```
library("readxl")
```

To load your own dataset, use the `read_data()` function, pointing it at the path to your file and specifying the format of the dates in your file's first column. For example, if you had a file called `"my_data.csv"` with dates formatted like `1/5/2017`, you would run:

```
Data_date <- read_data("my_data.csv", date_format = "%m/%d/%Y")

Data <- Data_date$Data
date <- Data_date$date
```

This returns a list with `Data` (a matrix of your variables) and `date` (a vector of the corresponding dates), ready to be used with `vars::VAR()` and the rest of the functions in this package.

Sample data used in this vignette

For replication, the remainder of this vignette uses datasets bundled with the package from Wiesen and Beaumont (2024), which contain the volatility of eleven major banks from three different periods: before the pandemic (January 5, 2017 through January 31, 2020), during the pandemic (February 3, 2020 through February 3, 2021), and over the full sample (January 5, 2017 through February 3, 2021). The banks used are Bank of America (BAC), Bank of New York Mellon (BK), Barclays (BCS), Citigroup (C), Credit Suisse (CS), Deutsche Bank (DB), Goldman Sachs (GS), JP Morgan Chase (JPM), Morgan Stanley (MS), State Street (STT), and Wells Fargo (WFC).

Load the pre-pandemic sample data as follows:

```
data(Volatility_data_pre)

Data <- Volatility_data_pre
date <- attr(Volatility_data_pre, "date")
```

Empirical Analysis

Impulse responses and variance decompositions

We refer users to Wiesen and Beaumont (2024) for a complete explanation of the methodology and its empirical contributions.

Estimate a VAR model

```
# Estimate VAR with 1 lag, as used in the paper
VAR_model <- VAR(Data, p = 1, type = "const")
```

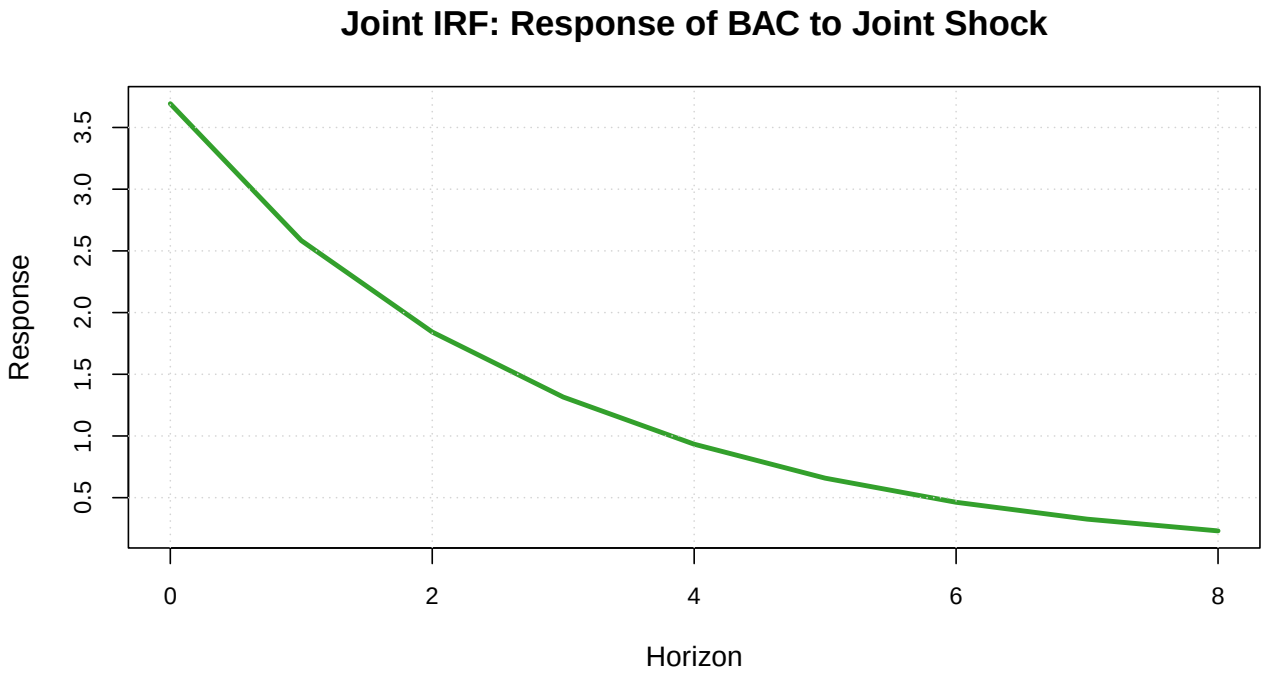
Joint Impulse Response Function (jIRF)

To compute the joint impulse response function involving shocks from Barclays (BCS), Credit Suisse (CS), and Deutsche Bank (DB), run this:

```
jIRF <- run_full_jIRF(VAR_model, from_set = c("BCS", "CS", "DB"), H = 8)
```

The following plot shows how Bank of America's equity volatility responds over horizons 0 through 8 trading days to simultaneous shocks of one standard deviation in Barclays, Credit Suisse, and Deutsche Bank. Since `jIRF` is a labeled matrix, you can plot it directly using base R:

```
horizon <- 0:8
plot(horizon, jIRF["BAC", ], type = "l", lwd = 3, col = "#33a02c",
     xlab = "Horizon", ylab = "Response",
     main = "Joint IRF: Response of BAC to Joint Shock",
     cex.main = 1.3, cex.lab = 1.1, cex.axis = 0.95)
abline(h = 0, col = "gray40", lty = 2)
grid()
```



Bank of America response to simultaneous shocks in Barclays, Credit Suisse, and Deutsche Bank.

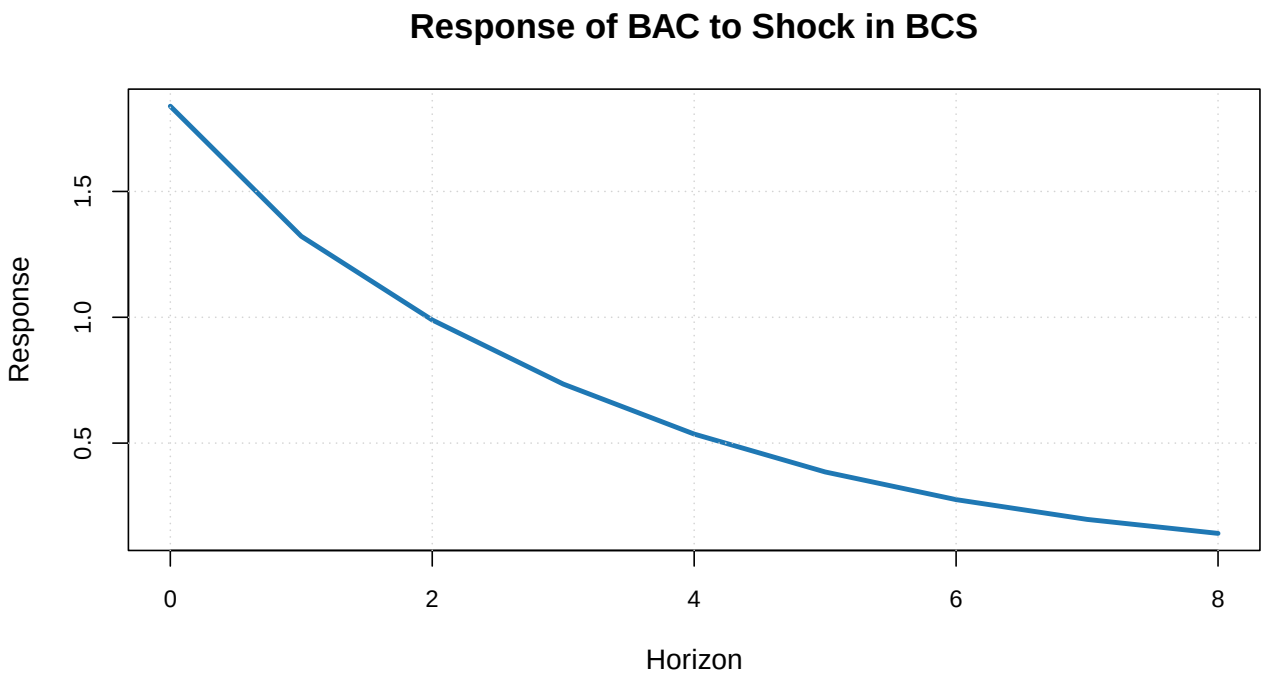
If you're interested in the gIRF of Pesaran and Shin (1998) without confidence bounds, run:

```
gIRF <- run_full_gIRF(VAR_model, H = 8)
```

This returns the entire set of gIRF results, showing how each variable responds to a one-standard-deviation shock in every other variable.

To visualize a specific response-shock pair, for instance, the effect of a shock from Barclays on Bank of America:

```
plot(horizon, gIRF["BAC", "BCS", ], type = "l", lwd = 3, col = "#1f78b4",
     xlab = "Horizon", ylab = "Response",
     main = "Response of BAC to Shock in BCS",
     cex.main = 1.3, cex.lab = 1.1, cex.axis = 0.95)
abline(h = 0, col = "gray40", lty = 2)
grid()
```

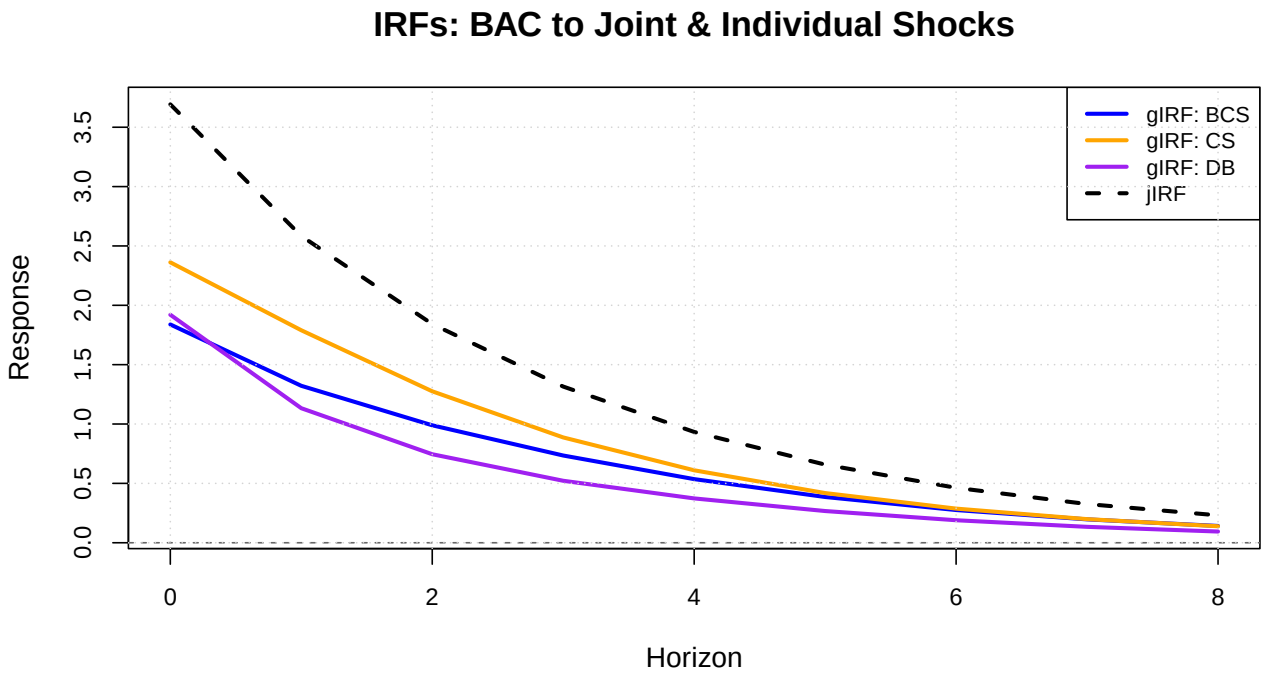


Bank of America response to a shock in Barclays.

You may compare the jIRF against individual gIRFs using:

```
from_set <- c("BCS", "CS", "DB")
g_colors <- c("blue", "orange", "purple")
j_color <- "black"
g_vals_list <- lapply(from_set, function(s) gIRF["BAC", s, ])
y_range <- range(c(unlist(g_vals_list), jIRF["BAC", ]))
plot(horizon, jIRF["BAC", ], type = "n",
     ylim = y_range, xlab = "Horizon", ylab = "Response",
     main = "IRFs: BAC to Joint & Individual Shocks",
     cex.main = 1.3, cex.lab = 1.1, cex.axis = 0.95)
for (i in seq_along(from_set)) {
  lines(horizon, g_vals_list[[i]], col = g_colors[i], lwd = 2.5, lty = 1)
}
lines(horizon, jIRF["BAC", ], col = j_color, lwd = 2.5, lty = 2)
abline(h = 0, col = "gray40", lty = 2)
legend("topright", legend = c(paste0("gIRF: ", from_set), "jIRF"),
```

```
       col = c(g_colors, j_color), lty = c(rep(1, length(from_set)), 2), lwd = 2.5, cex = 0.85)
grid()
```



Bank of America joint response compared with the individual generalized responses.

Note: To explore a different set of shock contributors, rerun `run_full_jIRF()` with a new `from_set` and recompute `jIRF`/`gIRF` before re-plotting.

Joint Forecast Error Variance Decomposition (jFEVD)

Users of this package can also calculate the joint forecast error variance decomposition (jFEVD) of Wiesen and Beaumont (2024). This jFEVD measures the proportion of a variable's forecast error variance that can be jointly explained by a specified set of multiple shocks.

We recommend referring to Wiesen and Beaumont (2024) for a detailed explanation of the model's theoretical advantages and empirical applications.

To illustrate, the jFEVD result can be interpreted as follows: approximately **16%** of the H=30 step forecast error variance of **Bank of America (BAC)** is attributable to joint shocks from **Barclays (BCS)**, **Credit Suisse (CS)**, and **Deutsche Bank (DB)**.

```
jfevd <- run_full_jFEVD(VAR_model, from_set = c("BCS", "CS", "DB"), H = 30)
jfevd
```

```
##         BAC          BK         BCS           C          CS          DB          GS
## 0.15952241 0.05223781 0.96668595 0.16460344 0.95693593 0.94891627 0.14332551
##         JPM          MS         STT         WFC
## 0.16926043 0.13025529 0.05798865 0.08725354
```

Generalized Forecast Error Variance Decomposition (gFEVD)

Another important concept is the **gFEVD** of Koop et al. (1996) and Pesaran and Shin (1998). This decomposition is embedded in the `genspill()` function and captures how shocks in one variable contribute to the forecast error variance of another.

While the `genspill()` function produces several outputs (discussed in the spillover section), our current interest is solely in the `gFEVD` matrix.

Each element (i, j) of the matrix quantifies the proportion of the H-step forecast error variance of variable i that can be attributed to shocks originating from variable j.
For instance, using pre-pandemic data, the gFEVD results suggest that **7.2%** of Bank of America's forecast error variance is explained by shocks from **Barclays (BCS)**, and about **6%** by shocks from **Deutsche Bank (DB)**.

```
genspill_results <- genspill(VAR_model, H = 30)
genspill_results$gFEVD
```

```
##            BAC         BK        BCS         C         CS         DB        GS
## BAC 0.9341628 0.19338713 0.07174584 0.6663017 0.11798916 0.06028896 0.5800812
## BK  0.3636745 0.84861996 0.02595194 0.3133308 0.03611692 0.01708104 0.3630035
## BCS 0.1105617 0.04713352 0.96118080 0.1137758 0.06482824 0.11523550 0.1256159
## C   0.5815652 0.18574850 0.08310222 0.9667632 0.11777901 0.05307041 0.4829849
## CS  0.1623523 0.05632249 0.05827780 0.1567158 0.95408863 0.20545501 0.1330114
## DB  0.1173690 0.04654549 0.08755388 0.1043196 0.21980032 0.93803018 0.1201977
## GS  0.5790122 0.16225563 0.08777056 0.5372057 0.08478607 0.04995554 0.9231996
## JPM 0.6204567 0.18944745 0.09349255 0.6724495 0.11208198 0.05924410 0.5367325
## MS  0.6003683 0.26112006 0.05947695 0.4635273 0.09129491 0.05815206 0.6050942
## STT 0.2704402 0.28513342 0.02768311 0.2650836 0.03344899 0.03398801 0.2973659
## WFC 0.4319118 0.16214739 0.04435500 0.4580954 0.05967095 0.03554170 0.3815070
##             JPM         MS        STT        WFC
## BAC 0.65819266 0.43345959 0.16944763 0.47917170
## BK  0.28284106 0.35771303 0.15529065 0.25641269
## BCS 0.12192095 0.09830710 0.05181303 0.06028663
## C   0.58618540 0.39354039 0.23290596 0.41561282
## CS  0.14284142 0.11647558 0.07622907 0.10152483
## DB  0.09594272 0.09836325 0.08129486 0.07489256
## GS  0.55648506 0.40694563 0.16974769 0.41660664
## JPM 0.93603194 0.37685117 0.19826444 0.55574753
## MS  0.43309480 0.84542878 0.15862593 0.34361573
## STT 0.19131216 0.33063710 0.79025181 0.16472681
## WFC 0.54340354 0.24595280 0.11973258 0.95833134
```

Bootstrapping

The interpretations provided above remain valid; however, bootstrapping allows us to compute **confidence intervals** around the impulse response functions, adding a layer of statistical inference.

Bootstrapping is split into four functions. First, `bootstrap_VAR()` bootstraps the VAR model itself, saving the constant, VAR coefficient matrices, and shock covariance matrix for each of `B` iterations. The first iteration is always the original (non-resampled) fitted parameters, so the original IRF can be recovered alongside the bootstrap distribution. Then, `bootstrap_gIRF()`, `bootstrap_jIRF()`, and `bootstrap_gIRF_sum()` each take the bootstrapped VAR parameters and compute confidence intervals for the generalized IRF, joint IRF, and sum of generalized IRFs, respectively, for whichever `response` variable you request.

Make sure you have sufficient memory available, as bootstrapping can be computationally intensive depending on the number of iterations (`B`). The default is `B = 10000`, and the default confidence interval is 90%, but you can set any desired level using the `CI` argument.

The examples below use 500 iterations to keep the tutorial quick to run and a fixed random seed for reproducibility. For empirical work, assess whether more iterations are needed for stable confidence bounds. All plotted confidence intervals below use `CI = 0.80`.

```
boot_model <- bootstrap_VAR(VAR_model, B = 500)
```

With `boot_model` computed once, we can now compute the bootstrapped confidence interval for the jIRF, the gIRF sum, or an individual gIRF, for any response variable, without re-running the bootstrap each time.

In the examples below, `response` selects the bank whose response is measured. Use `shocks` for a set of shock sources in the joint IRF or the sum of generalized IRFs, and `shock` for a single source in an individual generalized IRF.

Joint IRF (jIRF) with confidence interval

```
jIRF_CI <- bootstrap_jIRF(boot_model, response = "BAC", shocks = c("BCS", "CS", "DB"),
                          H = 8, CI = 0.80)
summary(jIRF_CI)
```

```
## jIRF: The response of BAC due to joint simultaneous shocks from BCS, CS, &
##
##     CI lower bound   mean median original CI upper bound stand. dev.
## h=0         3.2494 3.6592 3.6633   3.6927         4.0646      0.3146
## h=1         2.0748 2.5360 2.5381   2.5837         2.9931      0.3753
## h=2         1.3493 1.7937 1.7821   1.8423         2.2549      0.3641
## h=3         0.8940 1.2704 1.2646   1.3154         1.6541      0.3028
## h=4         0.5956 0.8940 0.8884   0.9334         1.1919      0.2379
## h=5         0.3972 0.6252 0.6169   0.6581         0.8575      0.1831
## h=6         0.2651 0.4363 0.4262   0.4627         0.6213      0.1403
## h=7         0.1742 0.3054 0.2979   0.3259         0.4456      0.1079
## h=8         0.1156 0.2153 0.2080   0.2307         0.3236      0.0834
```

Sum of generalized IRFs (gIRF sum) with confidence interval

```
gIRF_sum_CI <- bootstrap_gIRF_sum(boot_model, response = "BAC", shocks = c("BCS", "CS", "DB"),
                                  H = 8, CI = 0.80)
summary(gIRF_sum_CI)
```

```
## gIRF sum: The sum of the generalized responses of BAC due to individual
##
##     CI lower bound   mean median original CI upper bound stand. dev.
## h=0         5.2568 6.0866 6.0724   6.1212         6.9244      0.6250
## h=1         3.3481 4.1835 4.1819   4.2448         5.0479      0.6636
## h=2         2.1628 2.9434 2.9272   3.0103         3.7357      0.6245
## h=3         1.4410 2.0793 2.0487   2.1437         2.7196      0.5143
## h=4         0.9534 1.4618 1.4432   1.5194         1.9717      0.4022
## h=5         0.6337 1.0219 1.0030   1.0708         1.4075      0.3085
## h=6         0.4221 0.7129 0.6915   0.7525         1.0114      0.2357
## h=7         0.2767 0.4987 0.4854   0.5296         0.7292      0.1806
## h=8         0.1861 0.3512 0.3407   0.3747         0.5351      0.1392
```

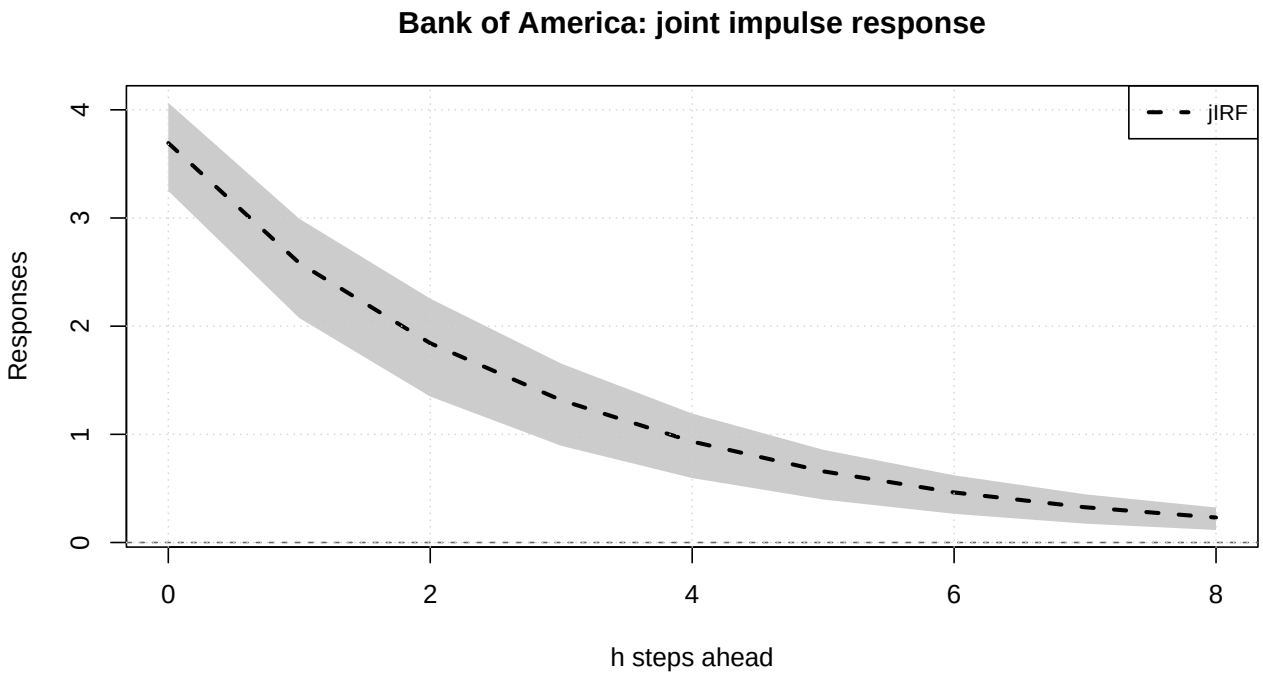
A single generalized IRF (gIRF) with confidence interval

```
gIRF_CI <- bootstrap_gIRF(boot_model, response = "BAC", shock = "BCS", H = 8, CI = 0.80)
summary(gIRF_CI)
```

```
## gIRF: The generalized response of BAC due to a shock from BCS.
##
##      CI lower bound   mean median original CI upper bound stand. dev.
## h=0          1.4998 1.8404 1.8279   1.8387          2.1793      0.2666
## h=1          0.9262 1.3099 1.2961   1.3219          1.7087      0.3001
## h=2          0.6249 0.9727 0.9722   0.9893          1.3395      0.2829
## h=3          0.4321 0.7161 0.7221   0.7347          1.0283      0.2298
## h=4          0.2943 0.5172 0.5164   0.5359          0.7577      0.1760
## h=5          0.2089 0.3682 0.3605   0.3854          0.5454      0.1320
## h=6          0.1410 0.2605 0.2548   0.2754          0.3943      0.0988
## h=7          0.0934 0.1846 0.1800   0.1968          0.2846      0.0744
## h=8          0.0620 0.1317 0.1292   0.1413          0.2069      0.0566
```

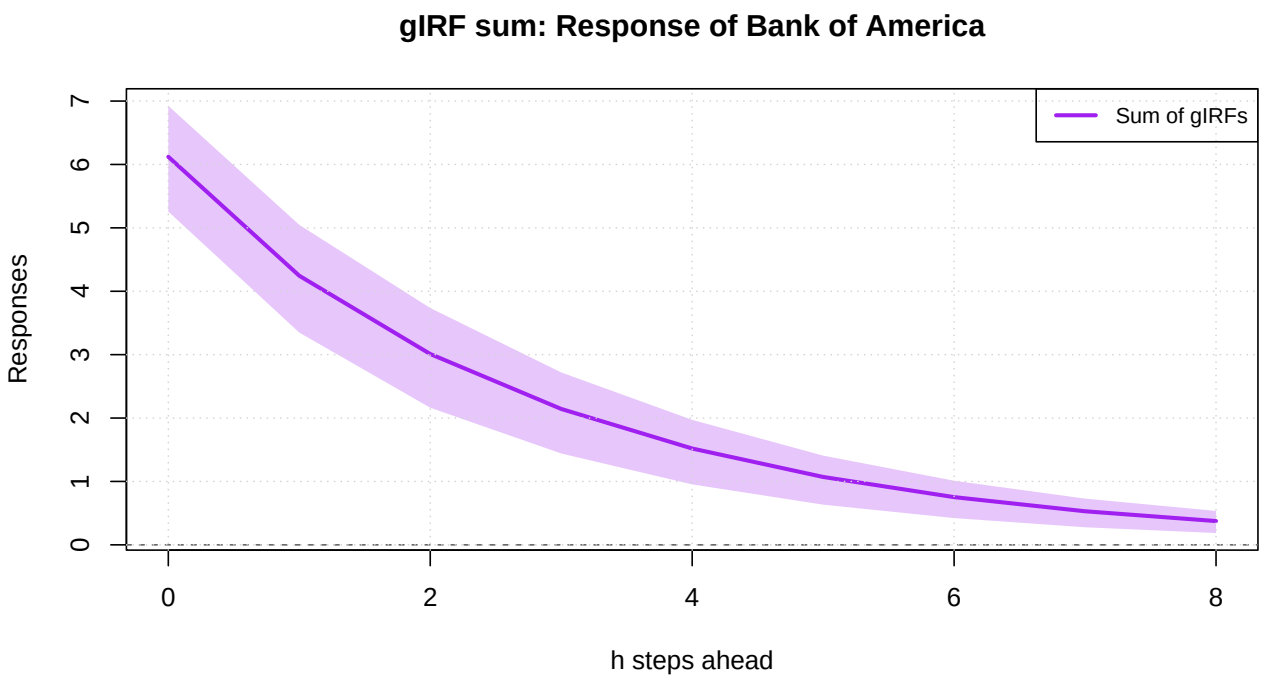
You can visualize any of these bootstrapped IRFs directly with base R's `plot()`/`polygon()`, using the `lower`, `original`, and `upper` fields returned by each function. In all of the examples below, the shaded region shows the 80% confidence interval, and the solid/dashed line shows the original (non-bootstrapped) IRF.

```r
horizon <- 0:jIRF_CI$H
plot(horizon, jIRF_CI$original, type = "n",
     ylim = range(c(jIRF_CI$lower, jIRF_CI$upper)),
     xlab = "h steps ahead", ylab = "Responses",
     main = "Bank of America: joint impulse response")
polygon(c(horizon, rev(horizon)), c(jIRF_CI$lower, rev(jIRF_CI$upper)),
        col = adjustcolor("black", alpha.f = 0.2), border = NA)
lines(horizon, jIRF_CI$original, col = "black", lwd = 2.5, lty = 2)
abline(h = 0, col = "gray40", lty = 2)
grid()
legend("topright", legend = "jIRF", col = "black", lty = 2, lwd = 2.5, cex = 0.85)
```



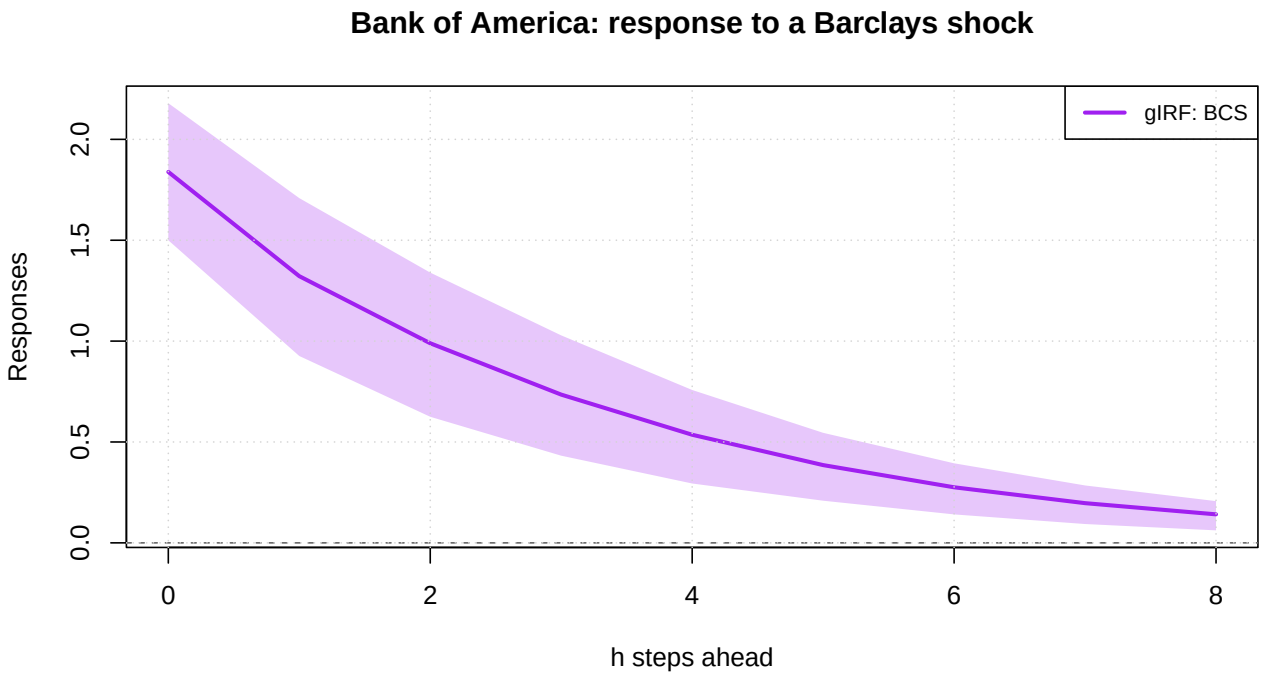
Bank of America joint impulse response with an 80% bootstrap confidence interval.

```
horizon <- 0:gIRF_sum_CI$H
plot(horizon, gIRF_sum_CI$original, type = "n",
     ylim = range(c(gIRF_sum_CI$lower, gIRF_sum_CI$upper)),
     xlab = "h steps ahead", ylab = "Responses",
     main = "gIRF sum: Response of Bank of America")
polygon(c(horizon, rev(horizon)), c(gIRF_sum_CI$lower, rev(gIRF_sum_CI$upper)),
        col = adjustcolor("purple", alpha.f = 0.25), border = NA)
lines(horizon, gIRF_sum_CI$original, col = "purple", lwd = 2.5, lty = 1)
abline(h = 0, col = "gray40", lty = 2)
grid()
legend("topright", legend = "Sum of gIRFs", col = "purple", lty = 1, lwd = 2.5, cex = 0.85)
```



Sum of Bank of America generalized responses with an 80% bootstrap confidence interval.

```
horizon <- 0:gIRF_CI$H
plot(horizon, gIRF_CI$original, type = "n",
     ylim = range(c(gIRF_CI$lower, gIRF_CI$upper)),
     xlab = "h steps ahead", ylab = "Responses",
     main = "Bank of America: response to a Barclays shock")
polygon(c(horizon, rev(horizon)), c(gIRF_CI$lower, rev(gIRF_CI$upper)),
        col = adjustcolor("purple", alpha.f = 0.25), border = NA)
lines(horizon, gIRF_CI$original, col = "purple", lwd = 2.5, lty = 1)
abline(h = 0, col = "gray40", lty = 2)
grid()
legend("topright", legend = "gIRF: BCS", col = "purple", lty = 1, lwd = 2.5, cex = 0.85)
```



Bank of America response to a Barclays shock with an 80% bootstrap confidence interval.

Replicating Figure 3 of Wiesen and Beaumont (2024)

We can replicate the pre-pandemic column of Figure 3 of Wiesen and Beaumont (2024), which shows the jIRF (with confidence interval), the three individual gIRFs, and the sum of gIRFs, due to joint shocks from Barclays (BCS), Credit Suisse (CS), and Deutsche Bank (DB), for a given response variable.

```
plot_fig3_panel <- function(boot_model, response, main) {
  shocks <- c("BCS", "CS", "DB")
  H <- 8
  CI <- 0.80
  horizon <- 0:H

  jIRF_CI <- bootstrap_jIRF(boot_model, response = response, shocks = shocks,
                            H = H, CI = CI)
  gIRF_sum_CI <- bootstrap_gIRF_sum(boot_model, response = response, shocks = shocks,
                                    H = H, CI = CI)
  gIRF_list <- lapply(shocks, function(s) {
    bootstrap_gIRF(boot_model, response = response, shock = s, H = H, CI = CI)
  })

  g_colors <- c("blue", "red", "darkgreen")
  g_ltys <- c(3, 2, 4)

  y_range <- range(c(jIRF_CI$lower, jIRF_CI$upper,
                     gIRF_sum_CI$lower, gIRF_sum_CI$upper,
                     sapply(gIRF_list, function(g) range(g$lower, g$upper))))

  plot(horizon, jIRF_CI$original, type = "n", xlim = c(0, H), ylim = y_range,
       xlab = "h steps ahead", ylab = "impulse response", main = main,
       cex.main = 1.1, cex.lab = 0.95, cex.axis = 0.9)

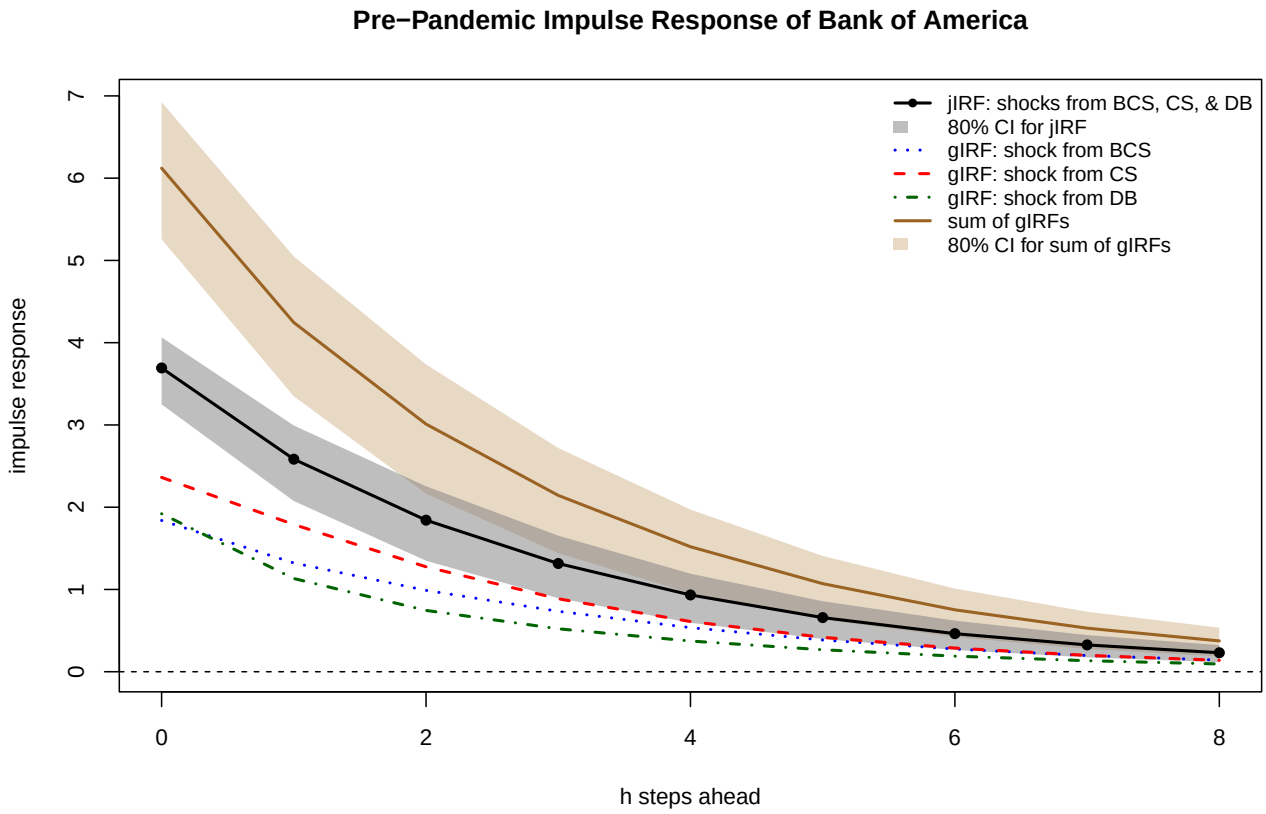
  polygon(c(horizon, rev(horizon)), c(gIRF_sum_CI$lower, rev(gIRF_sum_CI$upper)),
          col = adjustcolor("tan", alpha.f = 0.5), border = NA)
  polygon(c(horizon, rev(horizon)), c(jIRF_CI$lower, rev(jIRF_CI$upper)),
          col = adjustcolor("gray50", alpha.f = 0.5), border = NA)
```

```
  lines(horizon, gIRF_sum_CI$original, col = "#9A6324", lwd = 2)
  lines(horizon, jIRF_CI$original, col = "black", lwd = 2, type = "o", pch = 16)
  for (i in seq_along(shocks)) {
    lines(horizon, gIRF_list[[i]]$original, col = g_colors[i], lwd = 2, lty = g_ltys[i])
  }
  abline(h = 0, col = "black", lty = 2)

  legend("topright", cex = 0.8, bty = "n",
         legend = c("jIRF: shocks from BCS, CS, & DB", "80% CI for jIRF",
                    "gIRF: shock from BCS", "gIRF: shock from CS", "gIRF: shock from DB",
                    "sum of gIRFs", "80% CI for sum of gIRFs"),
         col = c("black", "gray50", g_colors, "#9A6324", "tan"),
         lty = c(1, NA, g_ltys, 1, NA),
         lwd = c(2, NA, 2, 2, 2, 2, NA),
         pch = c(16, NA, NA, NA, NA, NA, NA),
         fill = c(NA, adjustcolor("gray50", alpha.f = 0.5), NA, NA, NA, NA,
                  adjustcolor("tan", alpha.f = 0.5)),
         border = NA, merge = TRUE, seg.len = 2)
}
```

Now produce the first panel of Figure 3's pre-pandemic column, the pre-pandemic impulse response of Bank of America:

```
plot_fig3_panel(boot_model, "BAC", "Pre-Pandemic Impulse Response of Bank of America")
```



Pre pandemic Bank of America responses: joint IRF, individual generalized IRFs, and their sum. Shaded bands show 80% confidence intervals for the joint IRF and the sum.

The remaining panels of Figure 3 (Citigroup and JP Morgan Chase) can be reproduced the same way, by calling `plot_fig3_panel()` with `response = "C"` or `response = "JPM"`.

Spillover metrics

We refer users to the works of Lastrapes and Wiesen (2021) and Lastrapes and Wiesen (2025) to understand the concept of **joint spillover metrics**, and to Diebold and Yilmaz (2012) and Diebold and Yilmaz (2014) for the **generalized variant**.

As with the joint impulse response function and the joint forecast error variance decomposition discussed above, you must estimate a K-variable VAR model first before computing the (non rolling window) spillover metrics. As illustrated previously, say you want to load the pre-pandemic bank volatility data from Wiesen and Beaumont (2024). The dataset is described under "Sample data used in this vignette."

```
data(Volatility_data_pre)

Data <- Volatility_data_pre
date <- attr(Volatility_data_pre, "date")
```

```
VAR_model <- VAR(Data, p = 1, type = "const")
```

Once the VAR_model object above is available, the `jointspill()` function can be used to compute the joint spillover metrics.

The `jointspill()` function produces the following results:

1. The K x K **gFEVD matrix** (`gFEVD`)
2. The **joint total spillover from all others to variable *i*** (`jnt_from_others_to_i`)
3. The **joint total spillover from variable *i* to all others** (`jnt_to_others_from_i`)
4. The **joint net total spillover** (`jnt_net`)
5. The **joint spillover index** (`jSOI`)
6. The **lambda scaling factor** (`lambda`)
7. The **scaled gFEVD matrix** (`gFEVD_tilde`)

As before, `H` denotes the number of forecast periods ahead (i.e., the forecast horizon).

```
jointspill_results <- jointspill(VAR_model, H = 30)
```

If users want to extract any of the seven outputs calculated by `jointspill()`, they can use the `$` extraction operator. For example, `jointspill_results$gFEVD` returns the K x K generalized forecast error variance decomposition (gFEVD) matrix.

Additionally, users can easily view outputs (1) through (5) using the generic `summary()` function. This summary presents a well-structured table:

- The upper-left K x K portion contains the gFEVD matrix.
- The last column shows the **joint total spillover from all others** to each variable (*jnt_from_others_to_i*).
- The second-to-last row displays the **joint total spillover from each variable to all others** (*jnt_to_others_from_i*).
- The final row contains the **joint net total spillovers** (*jnt_net*).
- The **joint spillover index (jSOI)** is reported in the bottom-right corner of the table.

In the upper left matrix, rows identify the banks receiving shocks and columns identify the banks where shocks originate. Read an entry in row `BAC` and column `BK` as transmission from Bank of New York Mellon to Bank of America.

This summary can be generated using:

```
summary(jointspill_results)
```

Joint spillover summary

	BAC	BK	BCS	C	CS	DB	GS	JPM	MS	STT	WFC	From others to i
BAC	93.42	19.34	7.17	66.63	11.80	6.03	58.01	65.82	43.35	16.94	47.92	82.64
BK	36.37	84.86	2.60	31.33	3.61	1.71	36.30	28.28	35.77	15.53	25.64	53.04
BCS	11.06	4.71	96.12	11.38	6.48	11.52	12.56	12.19	9.83	5.18	6.03	22.56
C	58.16	18.57	8.31	96.68	11.78	5.31	48.30	58.62	39.35	23.29	41.56	73.27
CS	16.24	5.63	5.83	15.67	95.41	20.55	13.30	14.28	11.65	7.62	10.15	32.76
DB	11.74	4.65	8.76	10.43	21.98	93.80	12.02	9.59	9.84	8.13	7.49	33.40
GS	57.90	16.23	8.78	53.72	8.48	5.00	92.32	55.65	40.69	16.97	41.66	72.13
JPM	62.05	18.94	9.35	67.24	11.21	5.92	53.67	93.60	37.69	19.83	55.57	82.84
MS	60.04	26.11	5.95	46.35	9.13	5.82	60.51	43.31	84.54	15.86	34.36	75.28
STT	27.04	28.51	2.77	26.51	3.34	3.40	29.74	19.13	33.06	79.03	16.47	50.75
WFC	43.19	16.21	4.44	45.81	5.97	3.55	38.15	54.34	24.60	11.97	95.83	60.97
To others from i	95.07	39.37	15.84	92.92	23.23	17.04	89.82	89.48	70.81	35.01	71.06	jSOI
NET	12.43	-13.67	-6.72	19.65	-9.53	-16.36	17.68	6.64	-4.48	-15.74	10.09	58.15

As an example to interpret the results, the value in the first row and second column (the 1,2 element of the gFEVD matrix) implies that 19.34% of Bank of America's (BAC) forecast error variance can be explained by the shocks of Bank of New York Mellon (BK). The first value in the "FROM Others" column (the first element of **jnt_from_others_to_i**) implies that 82.64% of Bank of America's forecast error variance can be jointly explained by the shocks of the other ten banks in the model. The first value in the "TO Others" row (the first element of **jnt_to_others_from_i**) implies that the joint total spillover from Bank of America to the other ten banks is 95.07%. Note that this value can exceed 100%. The first value in the "Net" row (the first element of **jnt_net**) implies that the joint net total spillover for Bank of America is 12.43 percentage points. A positive net value identifies a net transmitter; a negative value identifies a net receiver. Bank of America is therefore a net transmitter in this example. Lastly, the value in the bottom-right is the joint spillover index (**jSOI**). This indicates that, on average, 58.15% of a bank's forecast error variance can be explained by jointly conditioning on the shocks of the other ten banks in the model.

Similar to the joint spillover results, the generalized spillover metrics of Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014) can also be estimated using the `genspill()` function.

This function returns the following outputs:

1. The K x K generalized spillover table (**gSOT**)
2. The generalized total spillover **from all others** to variable i (**gen_from_others_to_i**)
3. The generalized total spillover **from variable *i*** to all others (**gen_to_others_from_i**)
4. The **generalized net** total spillover (**gen_net**)
5. The **generalized spillover index** (**gSOI**)

You can compute these metrics by running:

```
genspill_results <- genspill(VAR_model, H = 30)
```

As in Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014), users can use the generic `summary()` function to generate a summarized version of the generalized spillover results. Rows again identify recipients and columns identify shock sources. The generalized spillover table applies normalization so that each row sums to 100%; its entries therefore differ from the unnormalized gFEVD percentages shown in the joint spillover summary.

- The **upper-left K x K** part of the table represents the **gSOT matrix**.
- The **last column** shows the generalized total spillover **from all others** to each variable.
- The **second-to-last row** displays the generalized total spillover **to all others** from each variable.
- The **last row** contains the generalized **net** total spillover.

- The **bottom-right** cell presents the overall **generalized spillover index (gSOI)**.

```
summary(genspill_results)
```

Generalized spillover summary

	BAC	BK	BCS	C	CS	DB	GS	JPM	MS	STT	WFC	From others to i
BAC	21.40	4.43	1.64	15.27	2.70	1.38	13.29	15.08	9.93	3.88	10.98	78.60
BK	12.04	28.10	0.86	10.38	1.20	0.57	12.02	9.37	11.84	5.14	8.49	71.90
BCS	5.91	2.52	51.38	6.08	3.47	6.16	6.72	6.52	5.26	2.77	3.22	48.62
C	14.19	4.53	2.03	23.58	2.87	1.29	11.78	14.30	9.60	5.68	10.14	76.42
CS	7.50	2.60	2.69	7.24	44.10	9.50	6.15	6.60	5.38	3.52	4.69	55.90
DB	5.91	2.35	4.41	5.26	11.08	47.27	6.06	4.84	4.96	4.10	3.77	52.73
GS	14.57	4.08	2.21	13.52	2.13	1.26	23.23	14.00	10.24	4.27	10.48	76.77
JPM	14.26	4.35	2.15	15.46	2.58	1.36	12.34	21.51	8.66	4.56	12.77	78.49
MS	15.32	6.66	1.52	11.83	2.33	1.48	15.44	11.05	21.57	4.05	8.77	78.43
STT	10.05	10.60	1.03	9.85	1.24	1.26	11.05	7.11	12.29	29.38	6.12	70.62
WFC	12.55	4.71	1.29	13.31	1.73	1.03	11.09	15.79	7.15	3.48	27.85	72.15
To others from i	112.31	46.84	19.83	108.19	31.33	25.30	105.93	104.66	85.31	41.45	79.45	gSOI
NET	33.72	-25.06	-28.79	31.78	-24.56	-27.43	29.16	26.17	6.88	-29.17	7.30	69.15

As an example to interpret the results, the value in the first row and second column (the 1,2 element of the gSOT matrix) shows the generalized bilateral spillover from Bank of New York Mellon (BK) to Bank of America (BAC) is 4.43%. Because of the row-sum normalization used by DY (Diebold and Yilmaz 2012; Diebold and Yılmaz 2014), this 4.43% is NOT the same thing as the percent of Bank of America's forecast error variance explained by the shocks of Bank of New York Mellon. The first value in the "FROM Others" column (the first element of gen_from_others_to_i) shows the generalized total spillover from the other ten banks to Bank of America is 78.60%. Because of the row-sum normalization and single-variable conditioning sets used by DY (Diebold and Yilmaz 2012; Diebold and Yılmaz 2014), this 78.60% is NOT the same thing as the percent of Bank of America's forecast error variance jointly explained by the shocks of the other ten banks in the model. The first value in the "TO Others" row (the first element of gen_to_others_from_i) shows the generalized total spillover from Bank of America to the other ten banks is 112.31%. The first value in the "Net" row (the first element of gen_net) implies that the generalized net total spillover for Bank of America is 33.72 percentage points. A positive generalized net value identifies a net transmitter and a negative value identifies a net receiver. Lastly, the value in the bottom-right is the generalized spillover index (gSOI) of 69.15%. Because of the row-sum normalization and single-variable conditioning sets used by DY (Diebold and Yilmaz 2012; Diebold and Yılmaz 2014), this 69.15% is not the same thing as the average percent of a bank's forecast error variance jointly explained by the shocks of the other ten banks in the model.

Rolling Window Analyses

To perform rolling window analyses for either the **joint spillover metrics** or its **generalized variant**, users must first estimate a rolling window VAR model. The parameter `win` specifies the window size, `p` specifies the VAR lag order, and `const = TRUE` includes an intercept. For example, the following line of code will estimate a rolling window VAR model of one lag and a window length of 150 periods using the pre-pandemic bank volatility data described above.

```
rolling_model <- VAR_roll(data = Data, p = 1, win = 150, const = TRUE, dates = date)
```

If users want to extract any of the four outputs produced by the `VAR_roll()` function, they can do so using the `$` extraction operator. The available components are:

1. `VARcoeff` - VAR coefficient matrices for each rolling window
2. `constant` - Intercepts for each rolling window
3. `Sigma` - Residual (shock) covariance matrices for each rolling window

4. `end_of_window_date` - The end-of-window date for each rolling window, computed from the `dates` and `win` inputs

For example, `rolling_model$VARcoeff` will return the rolling VAR coefficients.

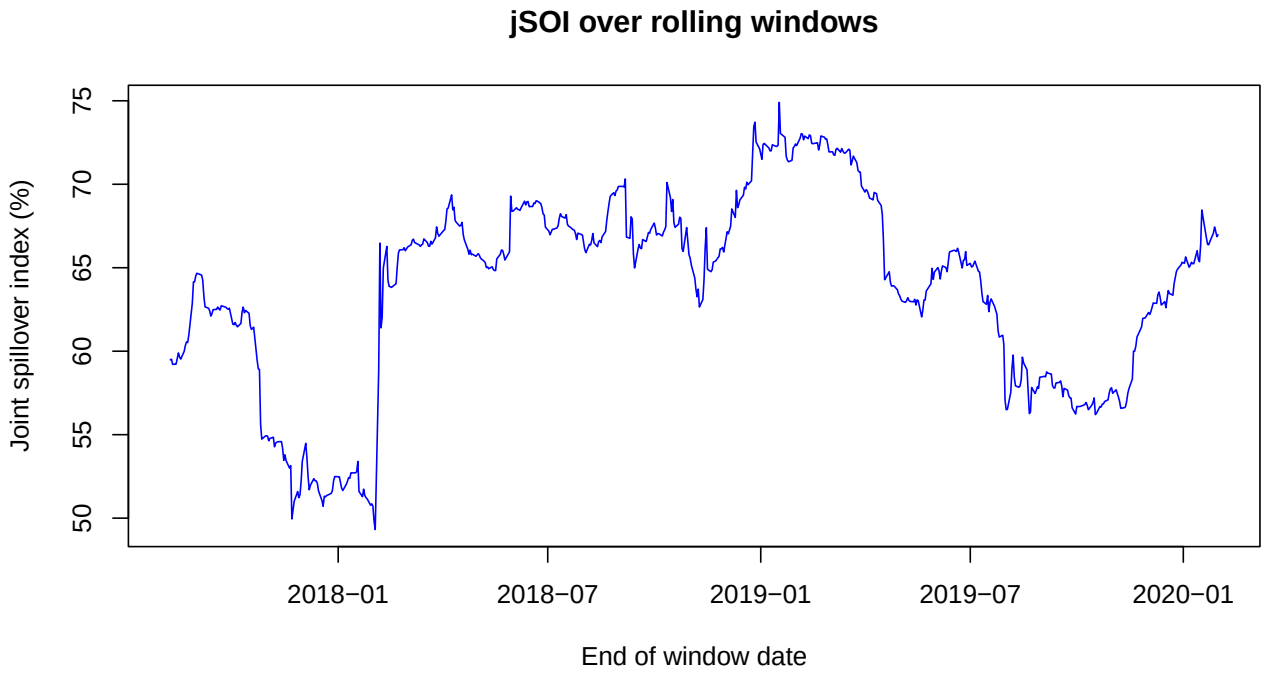
Compute the joint spillover results using the rolling window VAR model with a forecast horizon `H`. The `dates` input to `VAR_roll()` should be the date vector extracted earlier (`date <- attr(Volatility_data_pre, "date")`). The "date" input should not be truncated or shortened. The R code uses the `win` input together with `dates` to calculate the end-of-window date for each rolling window.

```
joint_spill_results_roll <- jointspill_roll(rolling_model, H = 30)
```

The rolling `gFEVD` output is expressed in percentages, whereas `genspill_results$gFEVD` above contains proportions. The spillover indices and directional spillovers are also expressed in percentages; net spillovers are differences in percentage points.

As an example, if users wish to plot the jSOI over rolling windows, they can run the following.

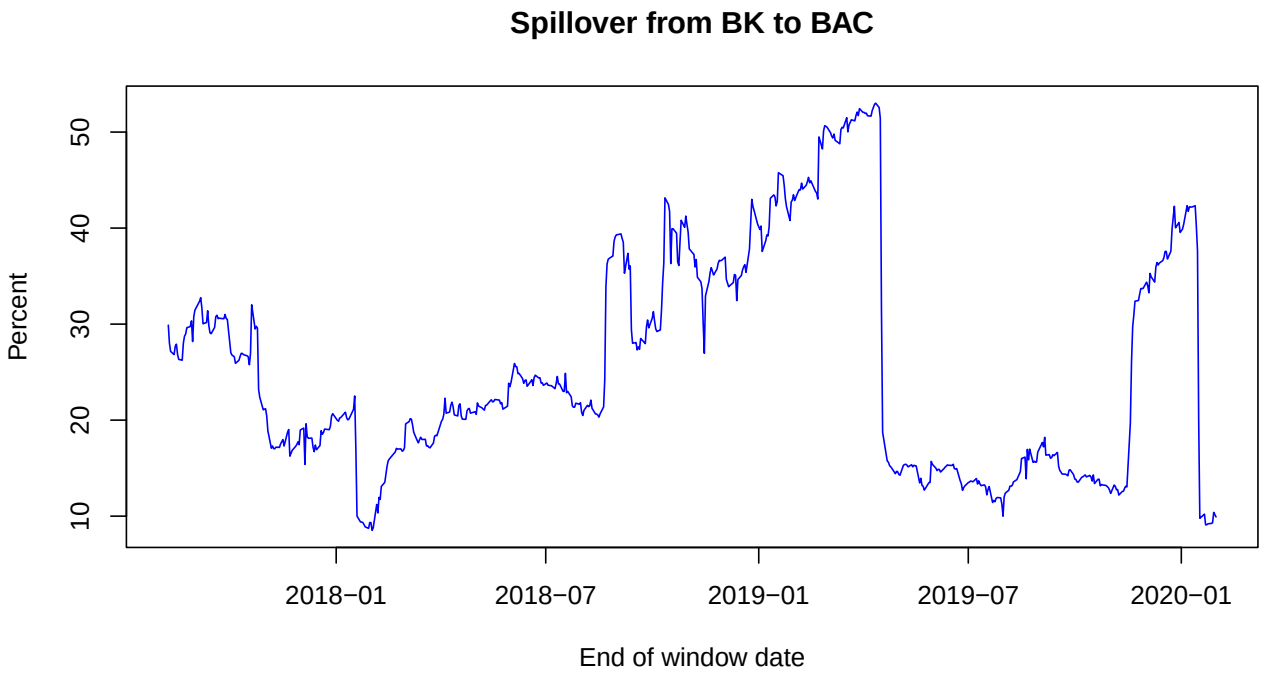
```
plot(joint_spill_results_roll$end_of_window_date, joint_spill_results_roll$jSOI,
     type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jSOI),
     xlab = "End of window date", ylab = "Joint spillover index (%)",
     main = "jSOI over rolling windows")
```



Joint spillover index over rolling windows of 150 observations.

If users wish to plot the 1,2 element of the gFEVD matrix (the percent of Bank of America's forecast error variance explained by the shocks of Bank of New York Mellon) over rolling windows, they can run the following

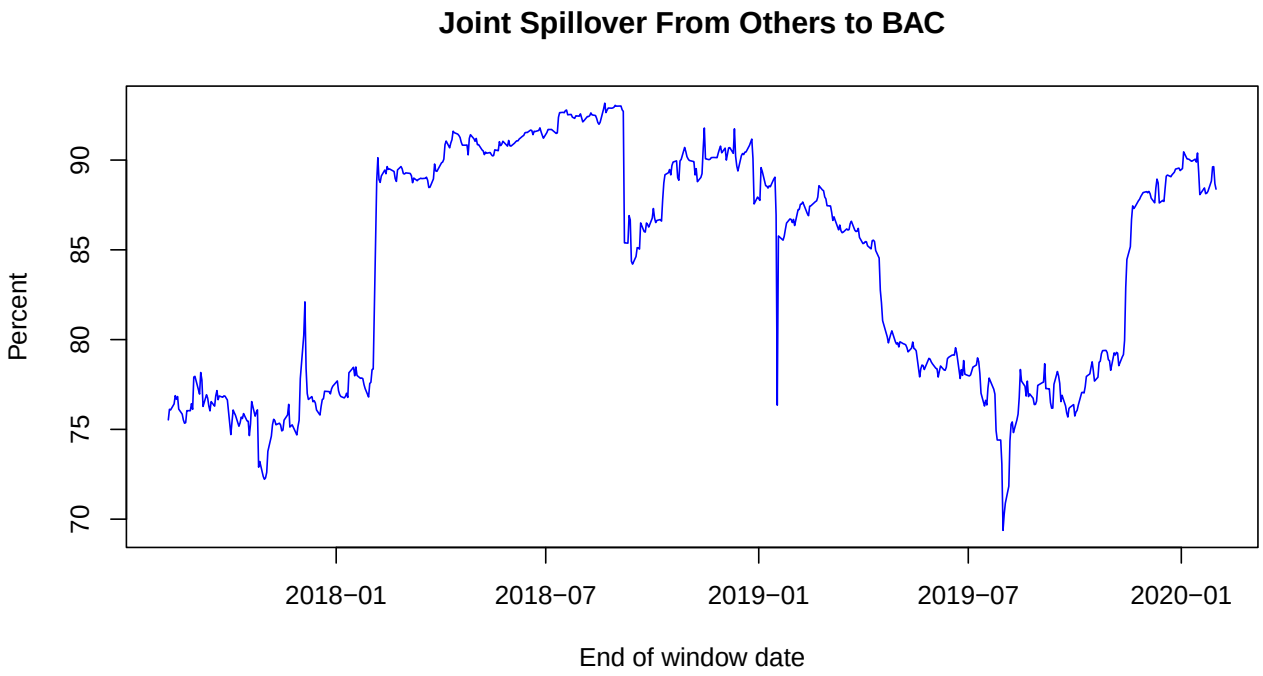
```
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$gFEVD["BAC","BK",], type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$gFEVD["BAC","BK",]),
     xlab = "End of window date", ylab = "Percent",
     main = "Spillover from BK to BAC")
```



Percentage of Bank of America forecast error variance explained by Bank of New York Mellon over rolling windows.

If users wish to plot the joint total spillover from all other banks to Bank of America over rolling windows, they can run the following.

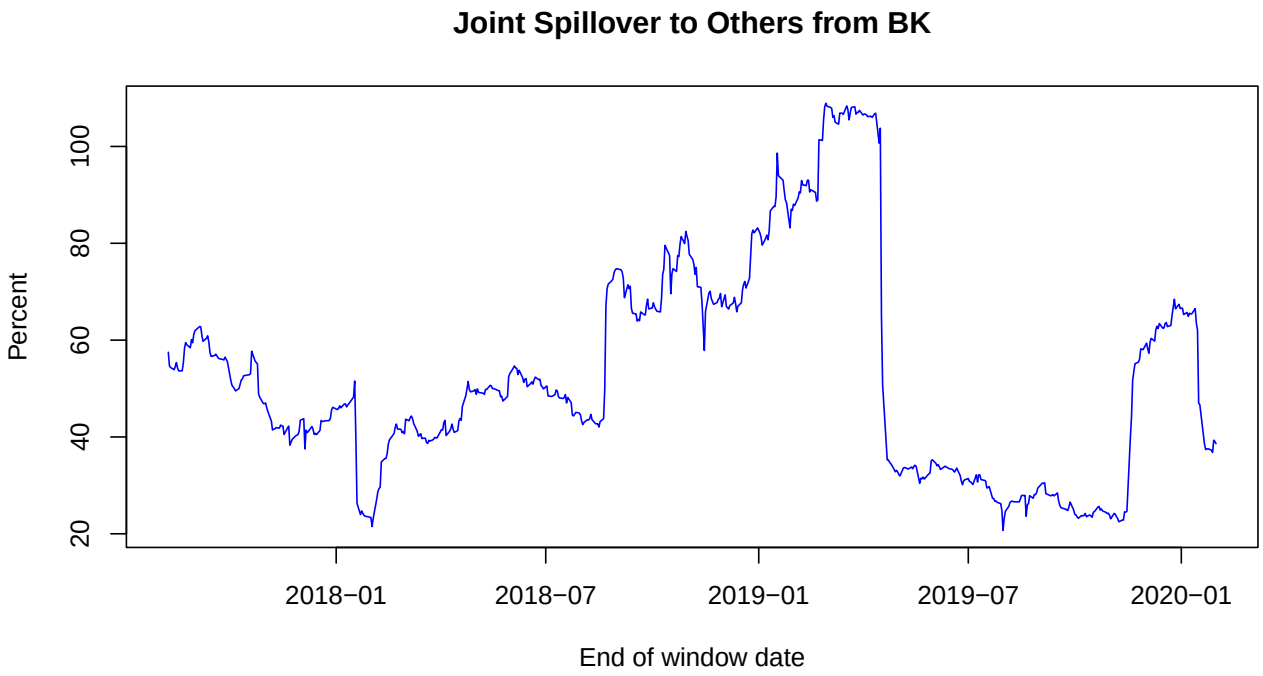
```
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jnt_from_others_to_i["BAC",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jnt_from_others_to_i["BAC",]),
     xlab = "End of window date", ylab = "Percent",
     main = "Joint Spillover From Others to BAC")
```



Joint spillover from all other banks to Bank of America over rolling windows.

If users wish to plot the joint total spillover from Bank of New York Mellon to the other banks over rolling windows, they can run the following.

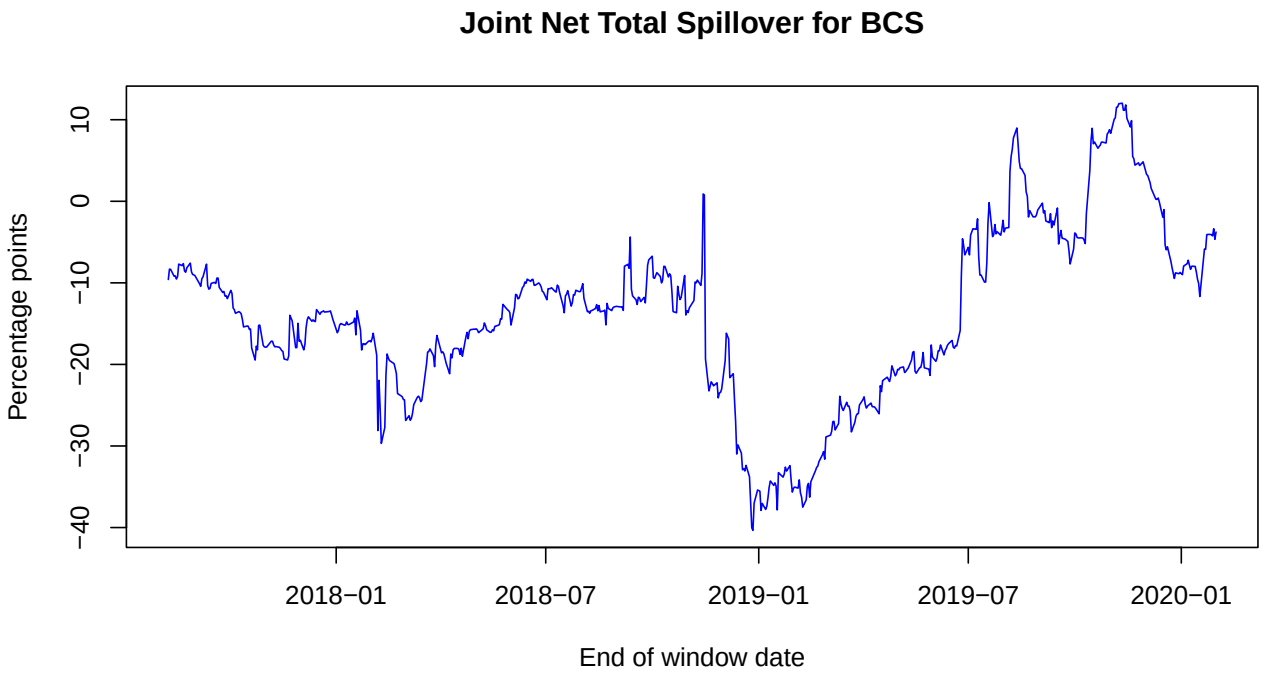
```
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jnt_to_others_from_i["BK",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jnt_to_others_from_i["BK",]),
     xlab = "End of window date", ylab = "Percent",
     main = "Joint Spillover to Others from BK")
```



Joint spillover from Bank of New York Mellon to the other banks over rolling windows.

If users wish to plot the joint net total spillover for Barclays over rolling windows, they can run the following.

```
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jnt_net["BCS",], type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jnt_net["BCS",]),
     xlab = "End of window date", ylab = "Percentage points",
     main = "Joint Net Total Spillover for BCS")
```



Joint net spillover for Barclays over rolling windows. Positive values indicate a net transmitter.

The command `genspill_roll()` can be used to compute the generalized spillover metrics over rolling windows. Notice that the arguments of the `genspill_roll()` command are:

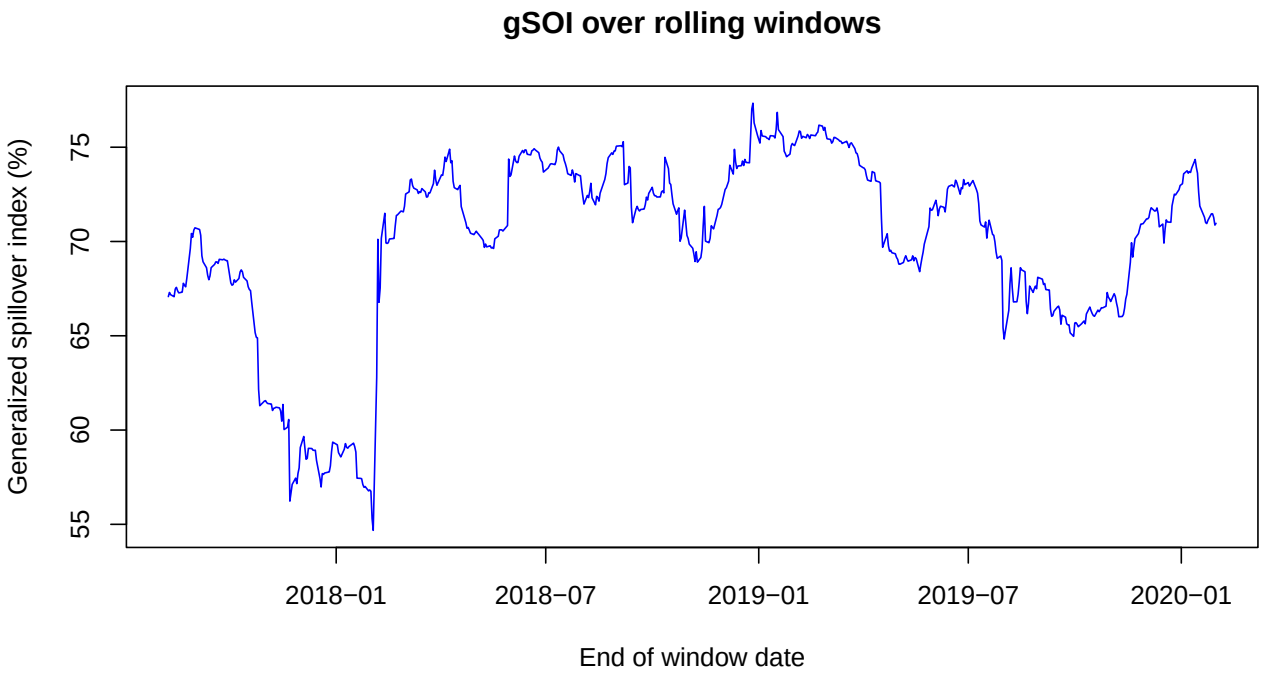
1. `rolling_model` - the rolling window VAR object
2. `H` - the forecast error horizon

Like `jointspill_roll()`, the end-of-window dates are carried through from `rolling_model`, so there is no separate `dates` argument.

```
gen_spill_result_roll <- genspill_roll(rolling_model, H = 30)
```

As an example, if users wish to plot the gSOI over rolling windows, they can run the following

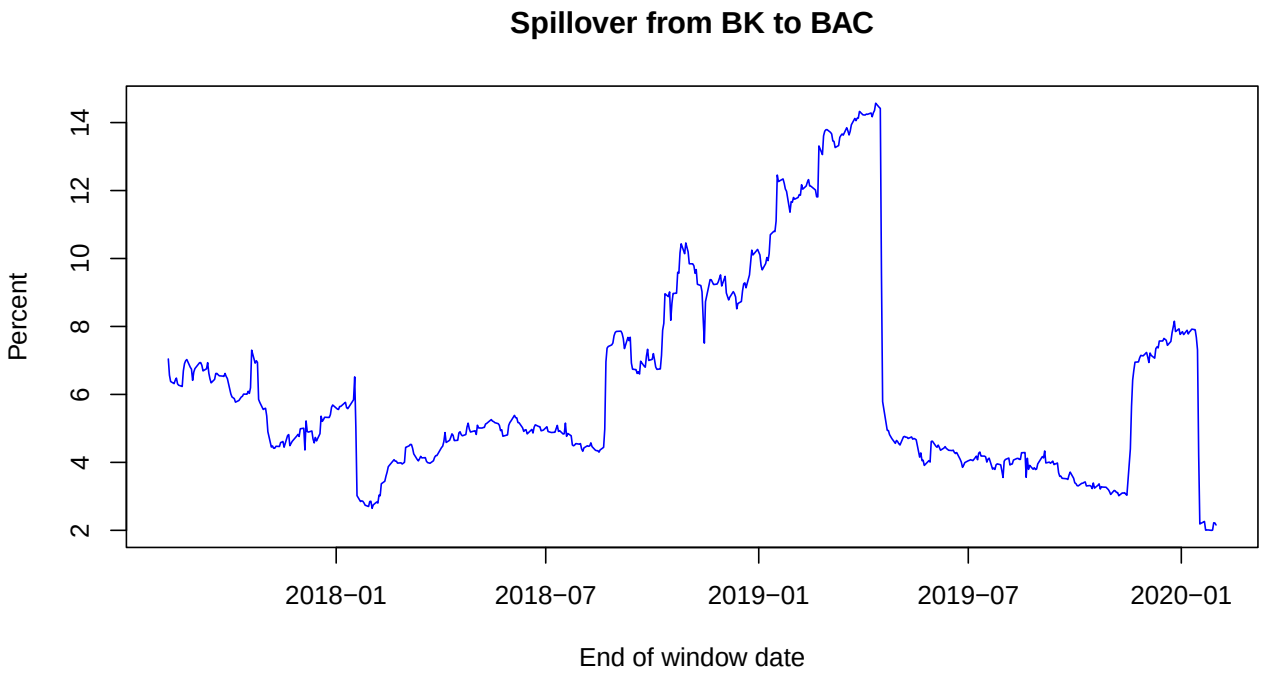
```
plot(gen_spill_result_roll$end_of_window_date, gen_spill_result_roll$gSOI,
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gSOI),
     xlab = "End of window date", ylab = "Generalized spillover index (%)",
     main = "gSOI over rolling windows")
```



Generalized spillover index over rolling windows of 150 observations.

If users wish to plot the 1,2 element of the gSOT matrix (generalized bilateral spillover to Bank of America from Bank of New York Mellon) over rolling windows, they can run the following

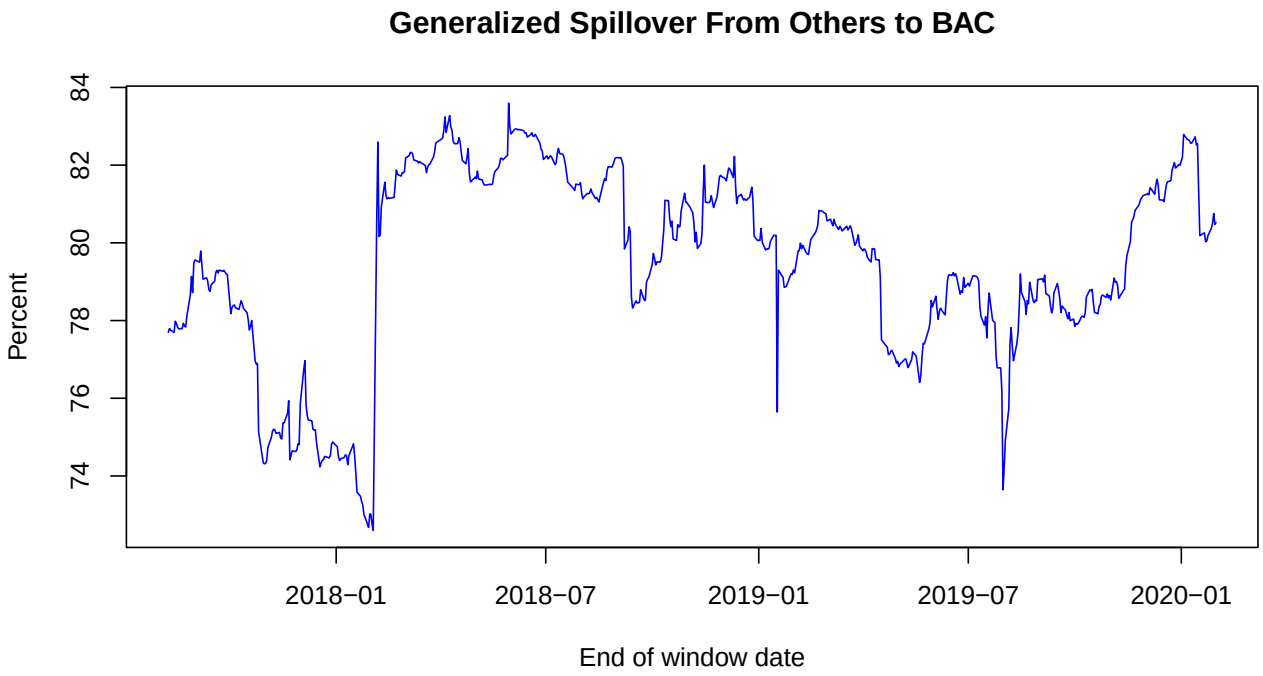
```
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gSOT["BAC","BK",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gSOT["BAC","BK",]),
     xlab = "End of window date", ylab = "Percent",
     main = "Spillover from BK to BAC")
```



Generalized bilateral spillover from Bank of New York Mellon to Bank of America over rolling windows.

If users wish to plot the generalized total spillover from all other banks to Bank of America over rolling windows, they can run the following.

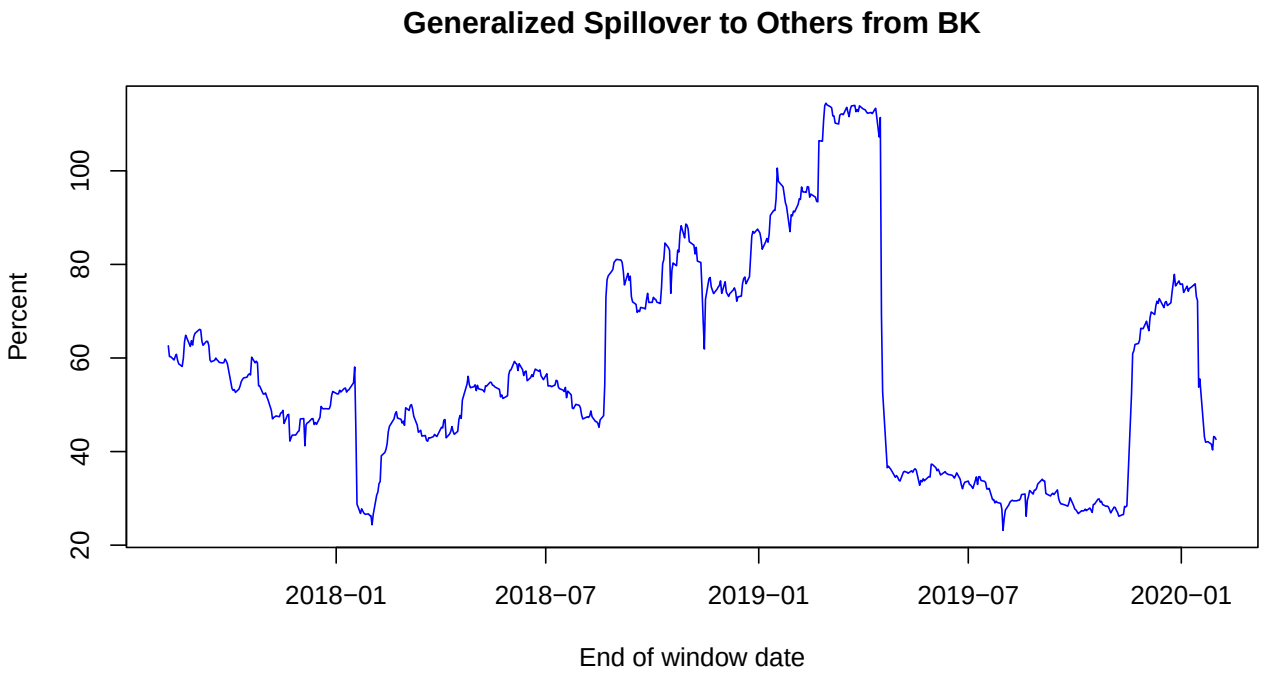
```
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gen_from_others_to_i["BAC",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gen_from_others_to_i["BAC",]),
     xlab = "End of window date", ylab = "Percent",
     main = "Generalized Spillover From Others to BAC")
```



Generalized spillover from all other banks to Bank of America over rolling windows.

If users wish to plot the generalized total spillover from Bank of New York Mellon to the other banks over rolling windows, they can run the following

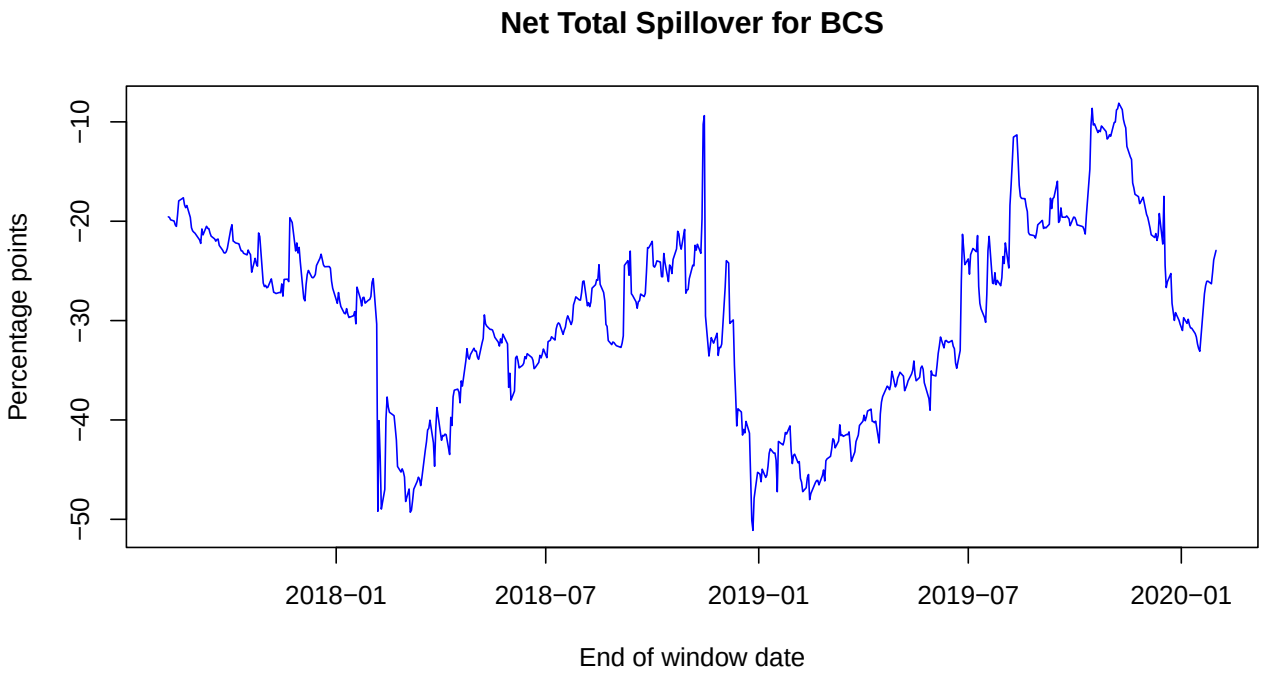
```
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gen_to_others_from_i["BK",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gen_to_others_from_i["BK",]),
     xlab = "End of window date", ylab = "Percent",
     main = "Generalized Spillover to Others from BK")
```



Generalized spillover from Bank of New York Mellon to the other banks over rolling windows.

If users wish to plot the generalized net total spillover for Barclays over rolling windows, they can run the following.

```
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gen_net["BCS",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gen_net["BCS",]),
     xlab = "End of window date", ylab = "Percentage points",
     main = "Net Total Spillover for BCS")
```



Generalized net spillover for Barclays over rolling windows. Positive values indicate a net transmitter.

Saving results

To export the joint and generalized spillover summary tables to CSV, run:

```
joint_table <- summary(jointspill_results)
write.csv(joint_table, "joint_spillovers.csv")

generalized_table <- summary(genspill_results)
write.csv(generalized_table, "generalized_spillovers.csv")
```

To save a figure as a PDF, open a graphics device, draw the plot, and close the device:

```
pdf("joint_spillover_index.pdf", width = 8, height = 5)
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jSOI,
     type = "l", col = "blue", lwd = 1,
     xlab = "End of window date",
     ylab = "Joint spillover index (%)",
     main = "Joint spillover index over rolling windows")
dev.off()
```

These files are saved in your current working directory, which you can check with `getwd()`. Supply a full file path to save them elsewhere. The examples above are shown for you to run when you wish to export results; they are not executed when this tutorial is rendered.

References

Diebold, Francis X, and Kamil Yilmaz. 2012. "Better to Give Than to Receive: Predictive Directional Measurement of Volatility Spillovers." *International Journal of Forecasting* 28 (1): 57–66.

Diebold, Francis X, and Kamil Yılmaz. 2014. "On the Network Topology of Variance Decompositions: Measuring the Connectedness of Financial Firms." *Journal of Econometrics* 182 (1): 119–34.

Koop, Gary, M Hashem Pesaran, and Simon M Potter. 1996. "Impulse Response Analysis in Nonlinear Multivariate Models." *Journal of Econometrics* 74 (1): 119–47.

Lastrapes, William D, and Thomas FP Wiesen. 2021. "The Joint Spillover Index." *Economic Modelling* 94: 681–91.

Lastrapes, William D, and Thomas FP Wiesen. 2025. "Regional Bank Failures and Volatility Transmission." *Journal of Financial Stability* 78: 101404.

Pesaran, H Hashem, and Yongcheol Shin. 1998. "Generalized Impulse Response Analysis in Linear Multivariate Models." *Economics Letters* 58 (1): 17–29.

Wiesen, Thomas FP, and Paul M Beaumont. 2024. "A Joint Impulse Response Function for Vector Autoregressive Models." *Empirical Economics* 66 (4): 1553–85.